סיכום שיעור 3: Join SQL- גל ששון

קורס: מבוא לבסיסי נתונים

השיעור עוסק בהיכרות עם שפת ה-SQL במיקוד על פקודת ה-JOIN ומסד הנתונים שאיתו נעבוד בקורס. להלן קישור למצגת השיעור

1. הסבר על פקודת ה- JOIN (חיבור בין טבלאות)

נשתמש ב join כשנרצה לשלוף נתונים השמורים בטבלאות שונות תחת אותה שאילתה. הצורך עולה כאשר קיים קשר לוגי בין הטבלאות (למשל: שחקן והסרטים שבהם השתתף), אך המידע מופרד כדי לשמור על סדר ולמנוע כפילויות.

פקודת ה JOIN = חיבור בין טבלאות. לרוב החיבור יהיה לפי עמודות משותפות (מפתחות), כדי להביא מידע שלא נמצא בטבלה אחת בלבד.

ON = נחפש key "מפתח" בעל ערכים משותפים לשתי הטבלאות שנחבר אותן לפיו.

תחביר כללי:

```
SELECT ...

FROM table1 t1

JOIN table2 t2

ON t1.column = t2.column
```

<u>דוגמה שבוצעה בכיתה בנושא:</u>

קישור לדוגמאות לשימוש בפקודת ה-join ותרגילי הכיתה שנעשו

- לדוגמה כאשר נרצה למצוא את שמות הסרטים שבהם משחק השחקן שון קונרי את התפקיד "ג'יימס בונד" .

בלי פקודת הjoin היינו צריכים לחפש ידנית בטבלת הactors את הactor id של שון קונרי ולאחר מכן לחפש בטבלת הroles את הmovie id של הסרטים בהם שון קונרי משחק את התפקיד ג'יימס בונד

ולבסוף בטבלת הmovies למצוא את שמות הסרטים התואמים לmovie id. כפי שניתן לראות זהו תהליך ארוך שאפשר לקצר ולפשט בעזרת פקודת join.

- בדוגמה הראשונה שולפים סרטים שיש בהם תפקיד של בונד.

```
use imdb_ijs;

select *

from roles

where role =  'James Bond';



select *

from actors

where first_name =  'sean' and

 last_name = 'connery';



select *

from roles

where actor_id = 93861 and `role`= 'James Bond'

;

# FInding James Bond movies “manually”

select *

from movies

where id in (85871 , 91948, 92573)
```

;

- מציאת כל סרטי ג׳יימס בונד (חיבור שתי טבלאות ביחד והצבת תנאי ששם הrole יהיה ג׳יימס בונד):

```
select *
from roles as r
join
movies as m
on
r.movie_id = m.id
where
r.role like 'james bond' ;
```

- מציאת כל הסרטים של שון קונרי:

```
select *
from
actors as a
join
roles as r
on
a.id = r.actor_id
join
movies as m
on
r.movie_id = m.id
where
a.first_name = 'sean' and a.last_name = 'connery';
```

- דוגמה לחיבור בין טבלת הroles לטבלת ה movies על מנת למצוא את הactor id של השחקנים ששיחקו את התפקידים בסרט ש movie id שלו זה 531

```
select *

from

roles as r

join

movies as m

on

r.movie_id = m.id

where

m.id = 531

;
```

2. **סוגי JOIN**

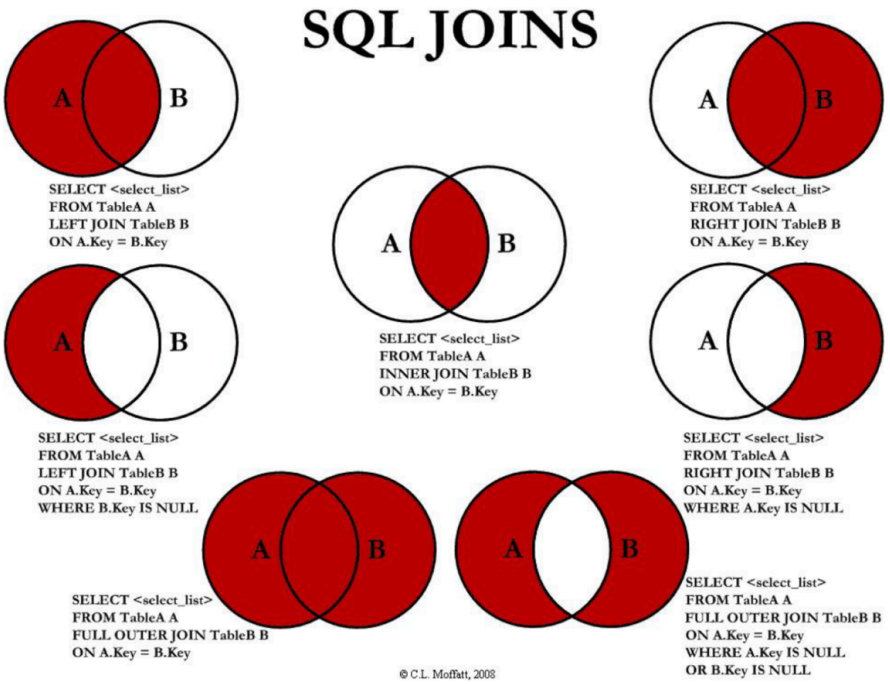
ניתן לחבר את הטבלאות במספר דרכים: inner join, left join, right join

מכיוון שSQL היא המקור שבהשראתה בנויות שפות תכנות אחרות, חיבור הטבלאות דומה לpandas בכך שיש חיבור פנימי (inner join), חיבור על פי טבלה שמאלית (left join) וחיבור על פי טבלה ימנית (right join).

<u>השוואה בין חיבור טבלאות בpandas ל- SQL</u>

Python (Pandas)	**SQL**	**שפת תכנות**
pd.merge(df1, df2)	JOIN	הפקודה
on='column_name'	ON	התנאי (על מה מחברים)
how='inner', 'left', 'right'	INNER, LEFT, RIGHT	סוג החיבור

sql joins as venn diagram



1.Inner join- join

מחזיר רק שורות שיש התאמה בשתי הטבלאות.

2.Left join

. נקבל את כל הטבלה השמאלית ונתונים מהטבלה הימנית איפה שיש התאמה. איפה שאין התאמה, נראה null.

דוגמה:

Directors and their movies (if exists)

Select *

```
from imdb_ijs.directors as d

left join

imdb_ijs.movies_directors as md

on

d.id = md.director_id ;
```

3. Right Join

בדיוק כמו ה-Left Join, רק הפוך. נקבל את השורות מהטבלה הימנית, ומהשמאלית רק מה שמתאים. השימוש בright join פחות נפוץ ולכן עדיף להימנע מהשימוש בו, שכן זה עלול להוביל לבלבול ושגיאות.

נדגיש שהשימוש בleft join ביחד עם right join עלול להוביל לבלבול וחוסר קריאות ולכן מהווה פתח לשגיאות. נעדיף תמיד להשתמש בleft join מבין השניים.

3. מספר נקודות / הערות נוספות שדיברנו עליהם בשיעור:

- אין בעיה לחבר יותר משתי טבלאות ביחד- פשוט מוסיפים עוד join. ב-SQL, חיבור של 3 טבלאות ומעלה עובד כמו שרשרת. לא מחברים את כל הטבלאות בבת אחת, אלא מחברים את הטבלה הראשונה לשנייה, ואז את "התוצאה המשותפת" שלהן מחברים לטבלה השלישית.

לדוגמה:

```
select *

from

actors as a

join

roles as r

on
```

```
a.id = r.actor_id

join

movies as m

on

r.movie_id = m.id

where

a.first_name = 'sean' and a.last_name = 'connery';
```

- נרצה לחבר את הטבלאות לפי הסדר של מה שאנחנו מחפשים למען הקריאות של הקוד. כדאי לומר בעברית מה אנחנו רוצים (תוך שימוש בטבלאות), הסדר הזה הוא לרוב הטבעי והיעיל ונותר רק לתרגם אותו ל sql.
- אפשר לעשות join לא לפי מפתח ולעשות join לטבלה עם עצמה. (לדוגמה על מנת למצוא זוגות של סרטים מאותו הבמאי)
- שימוש ב-as לצורך הגדרת שם לבחירתנו לטבלה (רלוונטי כשעושים self-join. חייבים להשתמש ב alias כדי להבדיל בין שני המופעים. במקרים כאלו כדאי שהשם ייצג את השימוש (לדוגמה, סרט אהוב סרט מומלץ).
- שימוש בlike: השימוש בlike רלוונטי רק כאשר אנו מחפשים ערכים שהם לא מדויקים. לא להשתמש בlike כברירת מחדל! (לא קשור לנושא join)

תרגול בכיתה

להלן דוגמה המראה את ההבדל בין סוגי החיבור

- בדוגמה זו נחבר את הטבלה להעתק של עצמה על מנת למצוא זוגות של סרטים מאותה השנה. חיבור זה רלוונטי כאשר נבנה את מערכת ההמלצות, נרצה להמליץ על סרטים שונים מאותה שנה במחשבה שמי שאהב סרט משנה כלשהי כנראה יאהב סרט אחר מאותה התקופה.

```
Select base.year , base.name, other.name
 from
imdb_ijs.movies as base
join
imdb_ijs.movies as other
on
base.year = other.year
 where
base.id < 1000 # for shorter run
And base.id != other.id # Avoid reflexivity
And base.id > other.id # Avoid symmetry ;
```

- מציאת כל הסרטים עם תפקיד של סופר מן (תרגיל כיתה מהמצגת):

```
select *
 from
 roles as r
 join
 movies as m
 on
```

```
r.movie_id = m.id
where
r.`role` = "superman"
;
```

- מציאת הסרטים של היצ׳קוק (director_id = 34658) וחיבור של זוגות סרטים שונים. חיבור זה רלוונטי כאשר נבנה את מערכת ההמלצות, נרצה להמליץ על סרטים שונים מאותו הבמאי במחשבה שמי שאהב סרט שבויים על ידי ביים מסויים כנראה יאהב סרט אחר מאותו במאי. (תרגיל כיתה מהמצגת)

```
select *
from
movies_directors as base
join
movies_directors as other
on base.director_id = other.director_id
where
base.director_id = 34658 and other.director_id = 34658
```

מטלת בית: אין חובת הגשה לתרגיל 3 בשיעורי הבית כרגע אך מומלץ לתרגל אותו.

פרוייקט: עברנו על התחלת הפרויקט. הפרוייקט יהיה בזוגות. בשלב הראשון נדרג 200 סרטים מתוך אוסף של סרטים פופולאריים. נעשה insert ונוסיף ציון 1-10 והצדקה ללמה הסרט קיבל את הציון. כרגע בשל המצב, אין תאריך הגשה, אך אפשר להתחיל לעבוד על זה. שימו לב ששלב זה מהווה 10% מהפרוייקט לכן צריך להמליץ על **200** סרטים, לא תתאפשר אפשרות להגיש מחדש. ראו סעיף personal ranking of movies בדף המטלה.